Universidad Nacional Autónoma de México

Facultad de Ingeniería

Asignatura: Programación Orientada a Objetos

Proyecto: Sistema Bancario en Java

Tarea de Investigación y Reporte Técnico - Versión 1

Equipo de Desarrollo:

- Francisco José Coutiño Morales (Núm. Cta: 425121623)
- Ernesto Flamenco Villaseñor (Núm. Cta: 319170599)
- Jaime Erick Torres Nava (Núm. Cta: 322289976)

Marzo 2026

Índice

1. Introducción **2**

2. Marco Teórico: Jerarquía de Colecciones en Java **2**
2.1. Clases Implementadas y Análisis de Complejidad 2
2.1.1. Listas (List) 2
2.1.2. Conjuntos (Set) 2
2.1.3. Tablas Hash (Map) 3
2.2. Tabla Comparativa de Estructuras 3
2.3. Criterios de Solución: Elección de la Colección 3

3. Arquitectura del Modelo de Datos (Aplicación) **4**
3.1. Clase Cliente 4
3.2. Clases de Cuentas 4

4. Implementación de Colecciones en el Sistema Bancario **4**
4.1. Uso de List (Historial de Movimientos) 4
4.2. Uso de Set (Cuentas del Cliente) 4
4.3. Uso de Map (Gestor del Banco) 4

5. Anexo: Fragmento de Código Clase de la Versión 1 **5**

6. Mejoras Arquitecturales Planificadas (Hacia la Versión 2) **6**
6.1. Automatización y Tipo de Dato de Identificadores 6
6.2. Módulo de Auditoría Global de Movimientos 6

7. Conclusiones **6**

1. Introducción

El presente documento tiene un doble propósito: presentar una investigación teórica sobre el *Java Collections Framework* y describir la arquitectura transaccional implementada para la primera versión de nuestro Sistema Bancario. El objetivo principal de esta fase fue modelar las entidades básicas del dominio bancario asegurando la integridad de los datos mediante las estructuras de datos nativas de Java.

2. Marco Teórico: Jerarquía de Colecciones en Java

El Java Collections Framework es una arquitectura para almacenar y manipular grupos de objetos de forma eficiente. Más allá de ser simples contenedores, estas estructuras están optimizadas a nivel de memoria y algoritmos de búsqueda. La jerarquía se divide principalmente en dos ramas:

- **Interfaz Collection:** De ella heredan las interfaces List y Set.
- **Interfaz Map:** Aunque no hereda directamente de Collection, forma parte esencial del framework al manejar pares clave-valor.

2.1. Clases Implementadas y Análisis de Complejidad

2.1.1. Listas (List)

Permiten elementos duplicados y mantienen un orden específico basado en la inserción. Clases que la implementan: `ArrayList` y `LinkedList`. A nivel interno, un `ArrayList` utiliza un arreglo dinámico que se redimensiona automáticamente, lo que permite un acceso a los datos en tiempo constante O(1). Por otro lado, `LinkedList` utiliza una estructura de nodos doblemente enlazados, optimizando las inserciones en los extremos. Sus principales métodos son:

- `add(E element)`: Agrega un elemento al final.
- `get(int index)`: Obtiene el elemento en una posición específica.
- `remove(int index)`: Elimina el elemento en la posición dada.
- `size()`: Retorna el número total de elementos.

2.1.2. Conjuntos (Set)

Estructuras que prohíben la existencia de elementos duplicados. Clases que la implementan: `HashSet` (rápido, sin orden) y `TreeSet` (mantiene orden alfabético o numérico). Internamente, un `HashSet` está respaldado por una tabla hash, ofreciendo operaciones matemáticas de conjuntos con gran rendimiento. Sus principales métodos son:

- `add(E e)`: Solo añade el objeto si no está ya presente.

- `contains(Object o)`: Verifica si un elemento existe en el conjunto.
- `isEmpty()`: Indica si el conjunto está vacío.
- `clear()`: Elimina todos los elementos.

2.1.3. Tablas Hash (Map)

Almacenan información mediante una relación de Çlave-Valor", donde cada clave debe ser única. Clases que la implementan: `HashMap` y `TreeMap`. Al utilizar funciones de *hashing*, mapean las llaves a çubetas"(buckets) en memoria, permitiendo búsquedas virtualmente instantáneas de complejidad O(1). Sus principales métodos son:

- `put(K key, V value)`: Guarda un valor asociado a una clave única.
- `get(Object key)`: Recupera el valor a través de su clave.
- `containsKey(Object key)`: Revisa si una clave ya está registrada.
- `values()`: Devuelve una colección de todos los valores guardados.

2.2. Tabla Comparativa de Estructuras

A continuación, se resumen las principales diferencias operativas de las interfaces:

Estructura	**¿Permite Duplicados?**	**¿Mantiene Orden?**	**Acceso a Datos**
List	Sí	Sí (por inserción)	Por índice (0, 1, 2...)
Set	No	No (generalmente)	Por iteración o búsqueda
Map	Solo en los valores	No	Por clave única

Cuadro 1: Diferencias entre List, Set y Map.

2.3. Criterios de Solución: Elección de la Colección

Para elegir la herramienta correcta al resolver un problema de programación, se deben considerar los siguientes puntos:

1. **Naturaleza de los datos:** Si necesitas un historial donde el orden de llegada importa, usa una Lista. Si los datos son únicos (como una lista de correos electrónicos), usa un conjunto.
2. **Necesidad de búsqueda:** Si vas a buscar elementos constantemente mediante un identificador (como buscar un cliente por su RFC), la Tabla Hash (Map) es la opción más rápida.
3. **Frecuencia de cambios:** Usa `ArrayList` si vas a leer datos frecuentemente. Usa `LinkedList` si vas a estar insertando o eliminando elementos en medio de la lista constantemente.

4. **Ordenamiento:** Si el problema exige que los datos estén siempre ordenados alfabéticamente mientras se agregan, se debe optar por versiones "Tree" (`TreeSet` o `TreeMap`).

3. Arquitectura del Modelo de Datos (Aplicación)

Basados en la investigación teórica previa, el sistema se construyó utilizando los principios de la Programación Orientada a Objetos (POO). Se diseñaron clases principales para representar la interacción entre los usuarios y sus activos.

3.1. Clase Cliente

Representa a los usuarios del banco. Cada cliente cuenta con un identificador único y actúa como contenedor de sus propias cuentas.

3.2. Clases de Cuentas

Se definieron dos tipos principales de cuentas:

- **Cuenta Básica:** Orientada a depósitos y retiros regulares.
- **Cuenta de Inversión:** Implementa lógica de cálculo de retornos basada en plazos y tasas de rendimiento, utilizando `java.time.LocalDate` para proyectar vencimientos.

4. Implementación de Colecciones en el Sistema Bancario

Para optimizar el rendimiento de la aplicación en memoria y cumplir con las reglas de negocio, se aplicaron los criterios de selección investigados:

4.1. Uso de List (Historial de Movimientos)

Se implementó un `ArrayList` dentro de las cuentas para registrar los objetos `Movimiento`. Se eligió esta colección porque permite preservar el orden cronológico exacto de las transacciones.

4.2. Uso de Set (Cuentas del Cliente)

Se utilizó un `HashSet` en la clase `Cliente` para almacenar sus cuentas. Esto garantiza la regla de negocio que prohíbe la duplicidad de productos financieros bajo un mismo titular.

4.3. Uso de Map (Gestor del Banco)

En la clase `GestorBanco`, se implementó un `HashMap` donde la llave es el número de cliente (`String`) y el valor es el objeto `Cliente`. Esto permite encontrar cualquier perfil de manera instantánea O(1).

5. Anexo: Fragmento de Código Clase de la Versión 1

A continuación se muestra cómo se implementó el mapa en el gestor para la búsqueda eficiente de clientes:

```
public class GestorBanco {
  // La llave es String (el numero de cliente actual)
  private Map<String, Cliente> directorioClientes;

  public GestorBanco() {
    this.directorioClientes = new HashMap<>();
  }

  public Cliente buscarCliente(String numCliente) {
    return directorioClientes.get(numCliente);
  }
}
```

Listing 1: Búsqueda Eficiente en GestorBanco

6. Mejoras Arquitecturales Planificadas (Hacia la Versión 2)

Tras el análisis de la Versión 1, el equipo ha definido los siguientes objetivos de reestructuración del código Java:

6.1. Automatización y Tipo de Dato de Identificadores

En la versión actual, el número de cuenta es introducido manualmente como una cadena de texto (`String`). Para la Versión 2:

- **Restricción Numérica:** El atributo `numCuenta` cambiará a tipo primitivo entero (`int`), garantizando de manera nativa que el sistema rechace caracteres alfabéticos.
- **Generación Automática:** Se implementará un módulo de generación aleatoria en el `GestorBanco`. Al registrar un cliente, el sistema asignará automáticamente un ID único.

6.2. Módulo de Auditoría Global de Movimientos

Para la Versión 2, se desarrollará un método administrativo en `GestorBanco` que permitirá iterar a través de todos los clientes y agregar el historial cronológico total de transacciones financieras institucionales.

7. Conclusiones

La investigación profunda del *Collections Framework* permitió elegir las estructuras de datos óptimas para el sistema bancario. La Versión 1 cumple exitosamente con los requisitos de encapsulamiento y asignación de estructuras. Los cambios planificados hacia la Versión 2 fortalecerán aún más la integridad del modelo financiero.